Enterprise Telemetry Report

Claude Code Usage Analytics

Insights from 60 Days of Enterprise Adoption

103.3M	$6,001	100	118K
Tokens Processed	Total API Cost	Active Engineers	API Requests

Section 1.1

Executive Summary: Key Findings

Core Metrics Dashboard

Metric	Value
Total Tokens Processed	**103.3 Million**
Total API Cost	**$6,001.43**
API Requests	**118,014**
Active Users	**100 engineers**
Total Sessions	**5,000**
Average Tokens/Request	**875**

Study Period

60 Days

Comprehensive telemetry analysis

Engineering Practices

5

ML, Frontend, Data, Backend, Platform

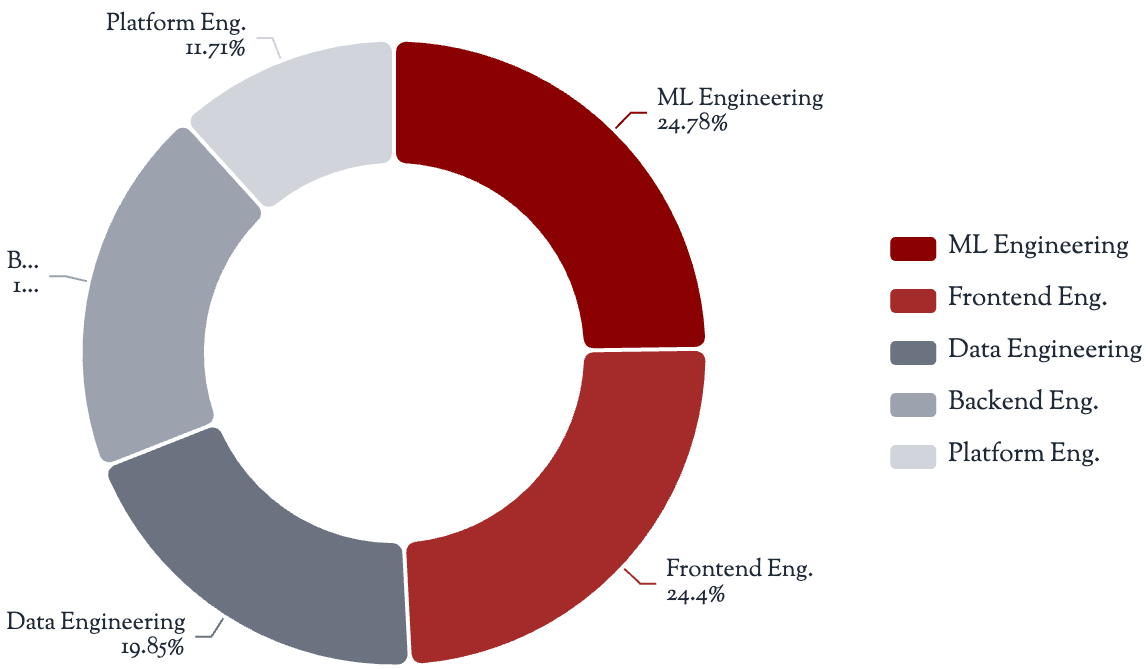
Key Narrative

ML Engineering and Frontend Engineering teams emerge as the heaviest users, collectively accounting for nearly **50% of total token consumption**. This concentration suggests these practices have found high-value integration patterns with Claude Code, particularly for code generation, refactoring, and architectural planning tasks. The 60-day adoption trajectory demonstrates strong enterprise penetration with consistent daily usage patterns.

Section 1.2

Token Consumption by Engineering Practice

Distribution Analysis



Consumption Breakdown

ML Engineering	25.6M (24.8%)
Frontend Engineering	25.2M (24.4%)
Data Engineering	20.5M (19.9%)
Backend Engineering	19.9M (19.3%)
Platform Engineering	12.1M (11.7%)

Strategic Insight

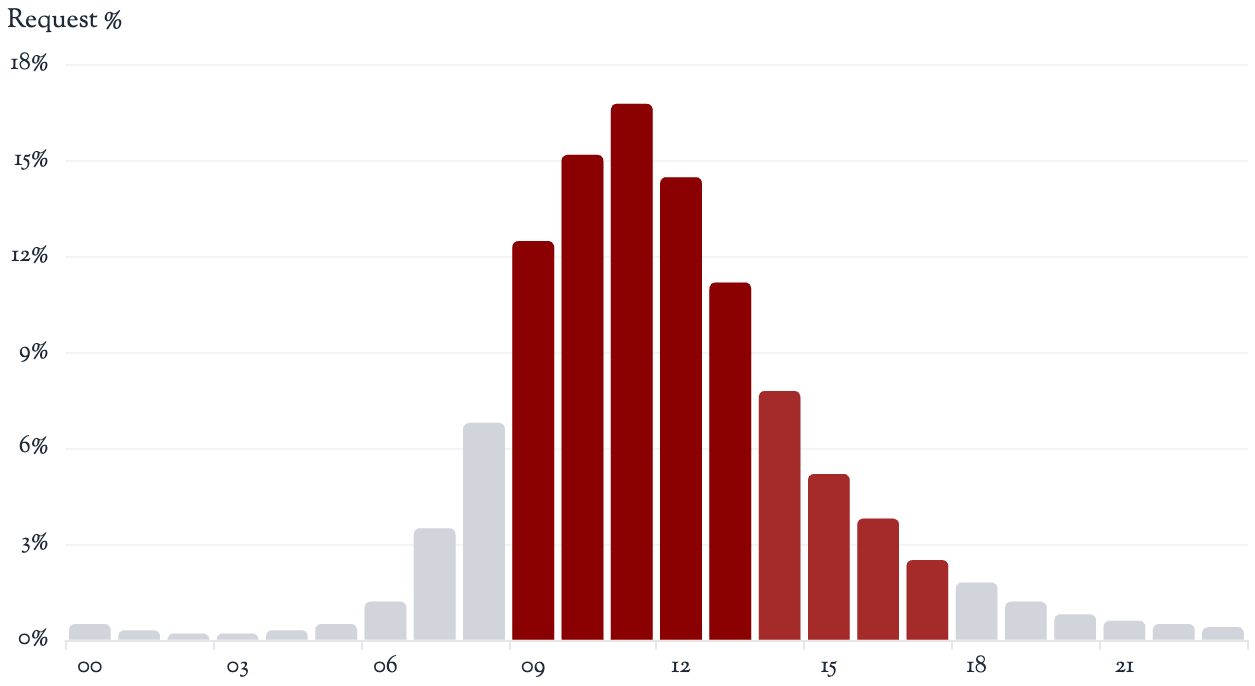
ML & Frontend teams consume nearly 50% of all tokens, indicating deep integration into their workflows. Platform Engineering shows lowest usage, presenting an opportunity for increased adoption and knowledge sharing.

Recommendation: Consider implementing tiered cost allocation by practice for budget planning, with ML and Frontend teams allocated proportionally higher budgets based on demonstrated usage patterns.

Section 1.3

Usage Patterns & Peak Hours Analysis

Hourly Distribution (UTC)



Temporal Insights

1. **Peak Hours**
   09:00-13:00 UTC accounts for **70% of all requests**
2. **Weekday Pattern**
   Tuesday-Thursday highest; Monday ramp-up; Friday decline
3. **Weekend Activity**
   Minimal usage (<5%), indicating work-focused adoption

Behavioral Observation

Engineers prefer Claude Code for focused morning work sessions, suggesting the tool is integrated into deep work periods rather than ad-hoc queries.

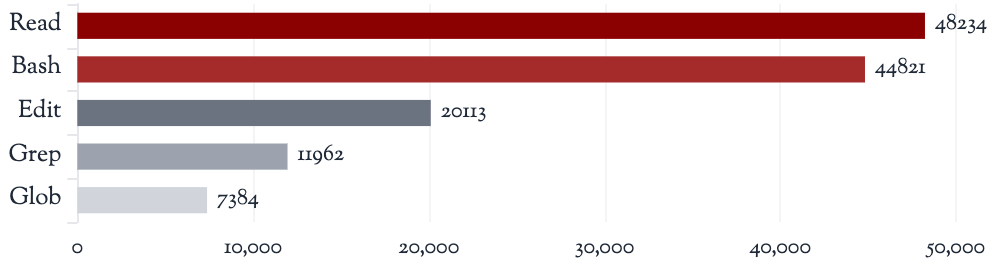
Morning Peak	Business Hours	Weekend Usage	Peak Day
70%	92%	<5%	Wednesday
of daily requests	of weekly usage	of total activity	highest engagement

Section 1.4

Tool Usage & Operational Efficiency

Tool Performance Metrics

Tool	Executions	Success Rate	Avg Duration
Read	48,234	**98.6%**	34 ms
Bash	44,821	**93.3%**	5,169 ms
Edit	20,113	**99.0%**	1,817 ms
Grep	11,962	**99.0%**	474 ms
Glob	7,384	**99.0%**	750 ms



Key Observations

- **Read & Bash dominate tool usage** with 63% combined executions, indicating core file system and command-line operations are primary use cases
- **Bash has the lowest success rate at 93.3%** - primarily due to command syntax errors and invalid paths
- **Edit operations are most reliable** at 99.0% success rate, reflecting the maturity of code editing workflows
- **WebSearch is rarely used** but has the longest average duration, suggesting complex query patterns

Strategic Recommendation

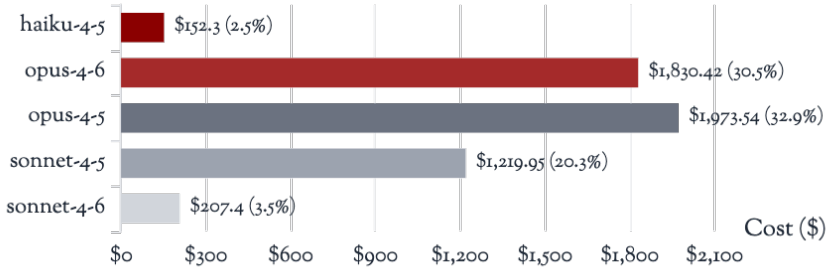
Provide targeted training on Bash command best practices to improve success rates. Consider implementing command validation and autocomplete features to reduce syntax errors.

Section 1.5

Cost Analysis & Model Distribution

Model Usage Breakdown

Model	Requests	Cost	$/1K Tokens
claude-haiku-4-5	45,934 (39%)	**$152.30**	$0.038
claude-opus-4-6	25,780 (22%)	**$1,830.42**	$0.25
claude-opus-4-5	23,487 (20%)	**$1,973.54**	$0.27
claude-sonnet-4-5	19,673 (17%)	**$1,219.95**	$0.18
claude-sonnet-4-6	3,140 (3%)	**$207.40**	$0.19



Cost Optimization Insights

Haiku Efficiency — Most Cost-Efficient

Haiku handles **39% of requests** but accounts for only **2.5% of total cost**. At $0.038 per 1K tokens, it's 6.6x cheaper than Sonnet.

Opus Cost Concentration — 63% of Total Cost

Despite only 42% of requests, Opus models (4-5 and 4-6) drive nearly **two-thirds of total expenditure** due to higher per-token pricing.

User Self-Selection — Appropriate Usage

Engineers demonstrate **effective model selection**, reserving expensive Opus models for complex reasoning tasks while using Haiku for routine operations.

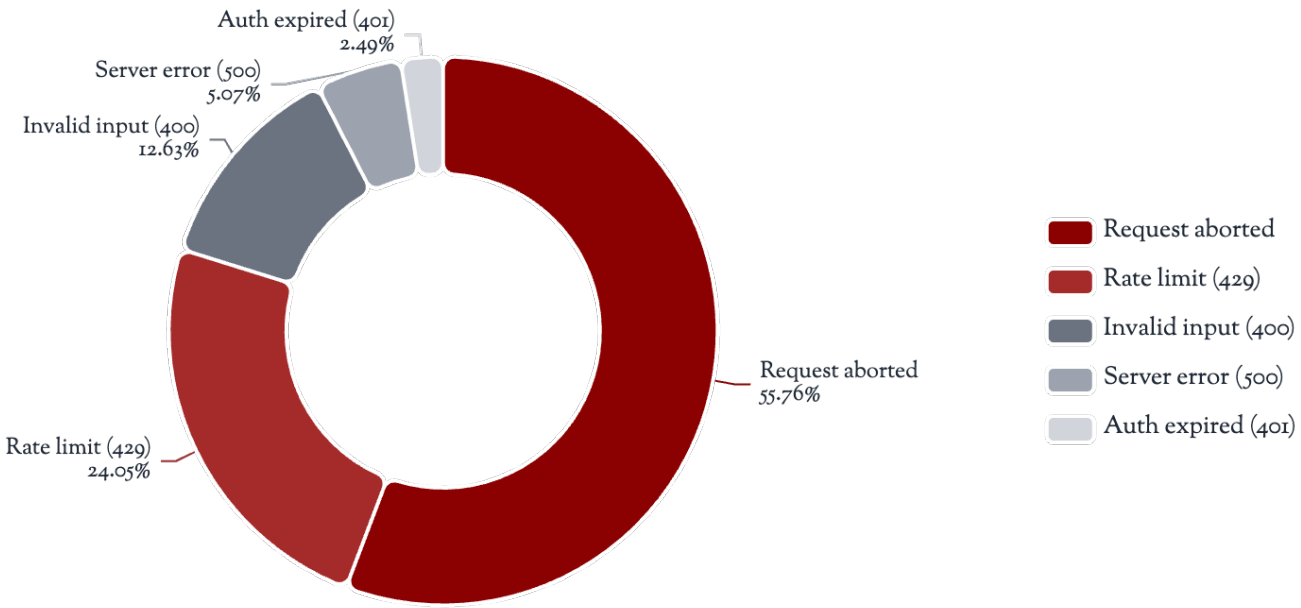
Strategic Recommendation

Encourage Haiku usage for simple tasks (code explanation, documentation, simple edits) and reserve Opus models for complex reasoning, architectural decisions, and debugging. Potential for **20% cost reduction** with optimized model selection.

Section 1.6

API Reliability & Error Analysis

Error Distribution



Error Breakdown

Error Type	Count	%
Request aborted	605	44%
Rate limit (429)	261	19%
Invalid input (400)	137	10%
Server error (500)	55	4%
Auth expired (401)	27	2%

Reliability Metrics

98.85%
Success Rate

1.15%
Error Rate

Operational Insights

Excellent Reliability

98.85% success rate demonstrates robust API performance and infrastructure stability across 118K requests.

Rate Limiting Impact

Rate limiting (19% of errors) primarily affects peak hours (09:00-13:00 UTC). Consider implementing request smoothing strategies.

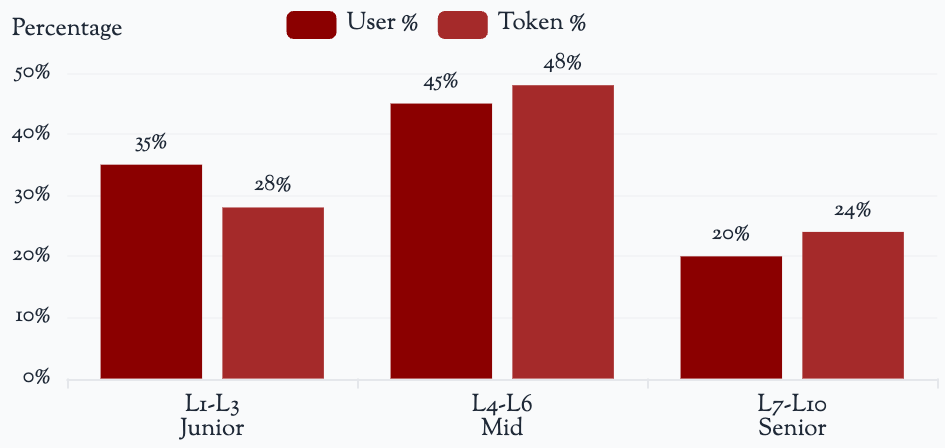
Authentication Issues

Auth errors (2% of total errors) indicate token refresh automation gaps. Implement proactive token renewal to reduce disruptions.

Section 1.7

User Segments & Geographic Distribution

Adoption by Seniority Level



L1-L3 Junior Engineers
35% of users, 28% of tokens

L4-L6 Mid-Level Engineers
45% of users, 48% of tokens

L7-L10 Senior Engineers
20% of users, 24% of tokens

Geographic Distribution

Location	Users	Token Share
Poland	28	26%
Germany	25	24%
United Kingdom	22	23%
Canada	15	16%
United States	10	11%

Power User Insight

Top 10% of users account for 40% of total token consumption, indicating a concentrated power user base. Mid-level engineers (L4-L6) show the highest adoption intensity, representing the core user segment.

Section 1.8

Strategic Recommendations & Next Steps

1 Cost Reduction

- **Promote Haiku for simple tasks** - Potential for **20% cost savings** with optimized model selection
- **Implement intelligent caching** for repeated queries and common code patterns
- **Set per-user daily token limits** with automatic alerts for budget management

2 Adoption Increase

- **Target Platform Engineering team** for focused onboarding - currently lowest adoption at 11.7%
- **Create Claude Code best practices documentation** with practice-specific use cases
- **Share power user workflows** through internal tech talks and knowledge sharing sessions

3 Performance Improvement

- **Implement Bash command validation** before execution to improve 93.3% success rate
- **Automate token refresh** to eliminate authentication errors (27 incidents)
- **Implement request queuing** during peak hours (09:00-13:00 UTC) to reduce rate limiting

4 Enhanced Monitoring

- **Real-time cost alerts by practice** with configurable thresholds and escalation
- **Weekly usage reports** to team leads with actionable insights and benchmarks
- **Anomaly detection** for unusual usage patterns and potential security issues

Expected Impact
20% cost reduction + 30% adoption increase in Platform Engineering

Timeline
Q1 2026 Implementation

Thank You

Dashboard Access

http://localhost:8501

Questions?

davitishkhanyan.01@gmail.com

Built with Python, DuckDB, Streamlit, and Plotly

60 days • 100 users • 454K events • 103.3M tokens

Data-driven insights for engineering excellence

2193247029